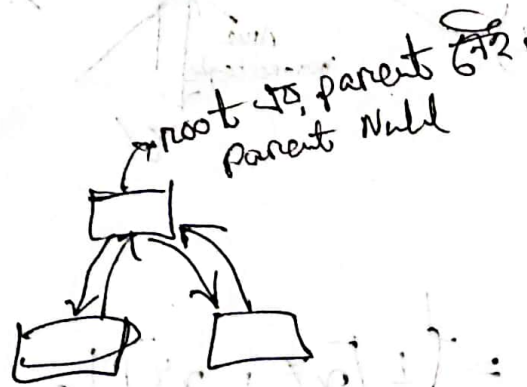
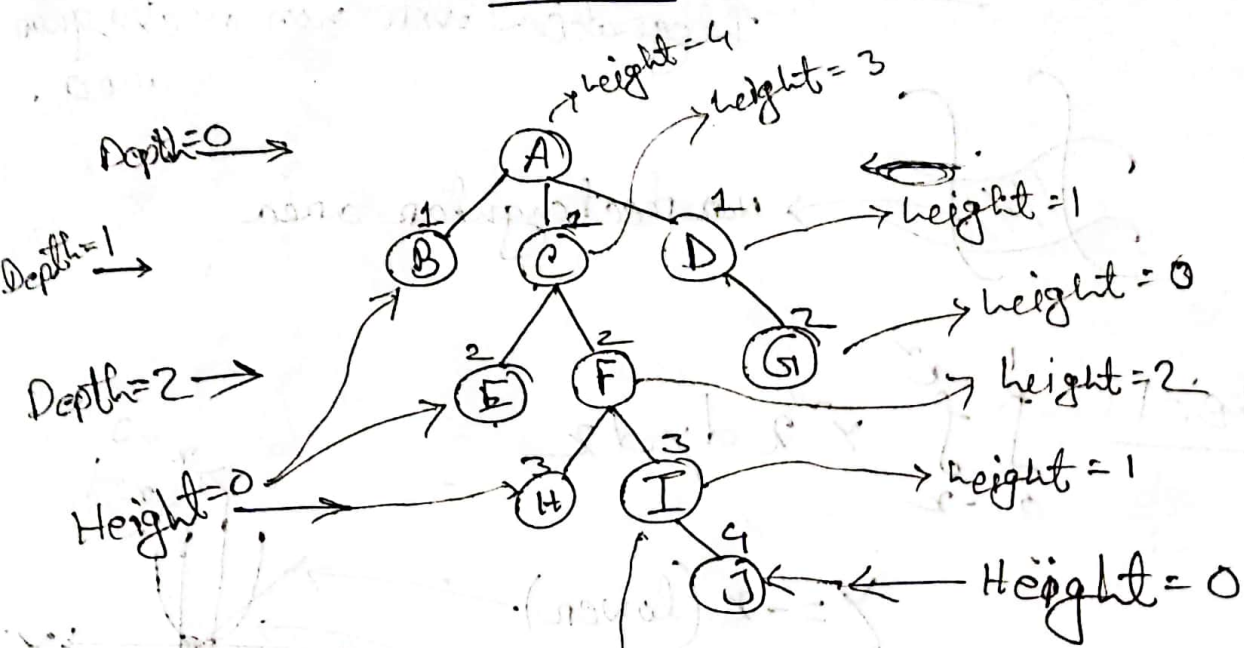


DSA-1

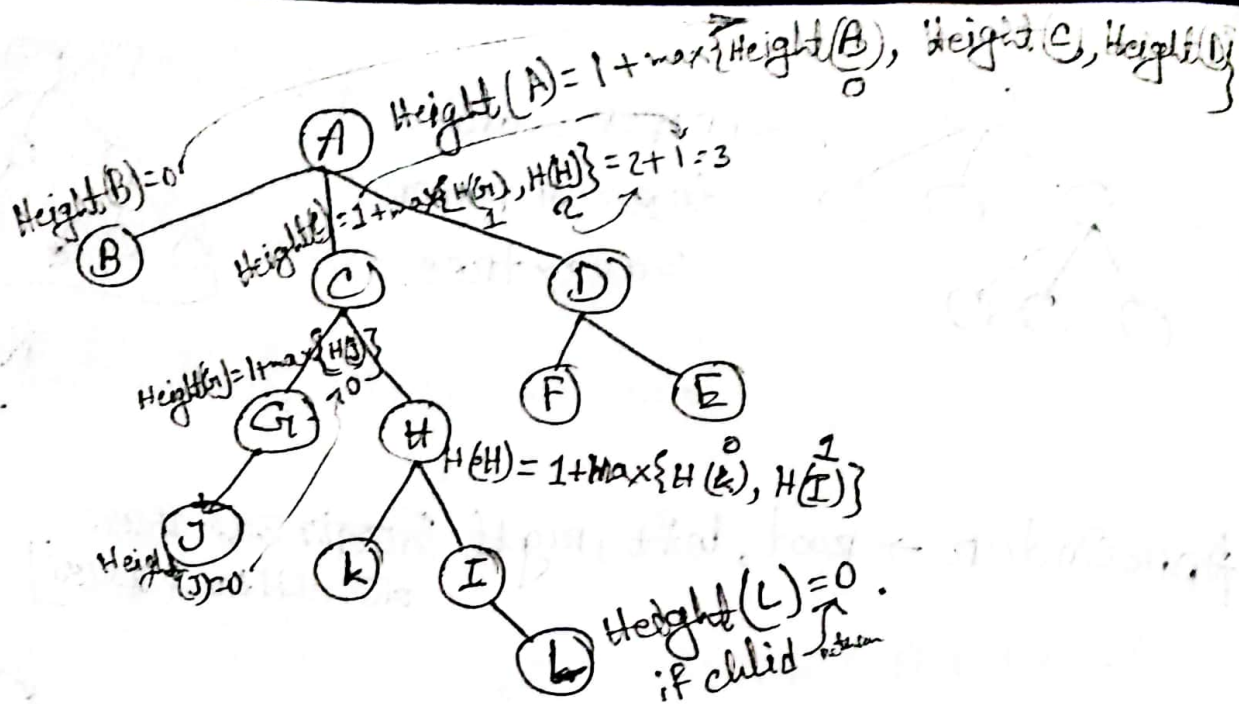


Depth(x)
 ↳ root is check
 ↳ Depth is

$$\text{Depth}(I) = 1 + \text{Depth}(F)$$

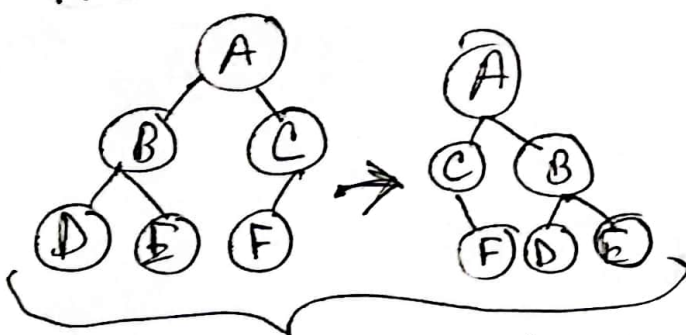
*root is operation
 ↳ parent

*



Ordered Tree

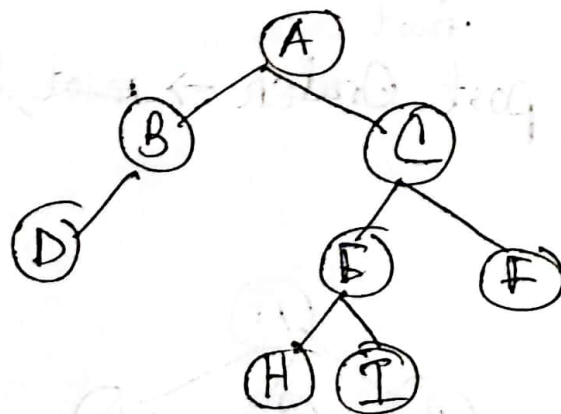
*



unordered tree

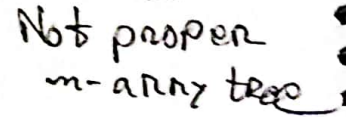
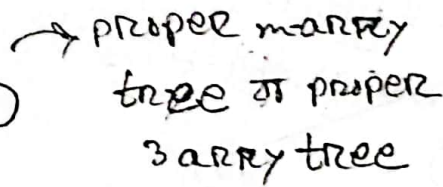
આ order બદલી શકાય
order change થઈ શકે,
આ ordered tree નો
જોઈ order change થઈ
શકે નહીં.

Binary tree is
a ordered tree



Height 2 for
children 288

proper max tree રજા,
બાકી 288 ના 388 ---
children 288,

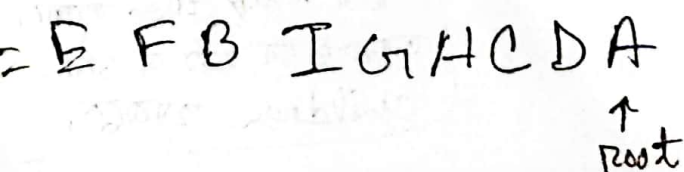


```

graph TD
    A((A)) --- B((B))
    A --- C((C))
    A --- D((D))
    B --- E((E))
    B --- F((F))
    C --- G((G))
    C --- H((H))
    C --- I((I))
    style E stroke-dasharray: 5 5
    style F stroke-dasharray: 5 5
    style G stroke-dasharray: 5 5
    style H stroke-dasharray: 5 5
    style I stroke-dasharray: 5 5
  
```

visit [console](#)
print array

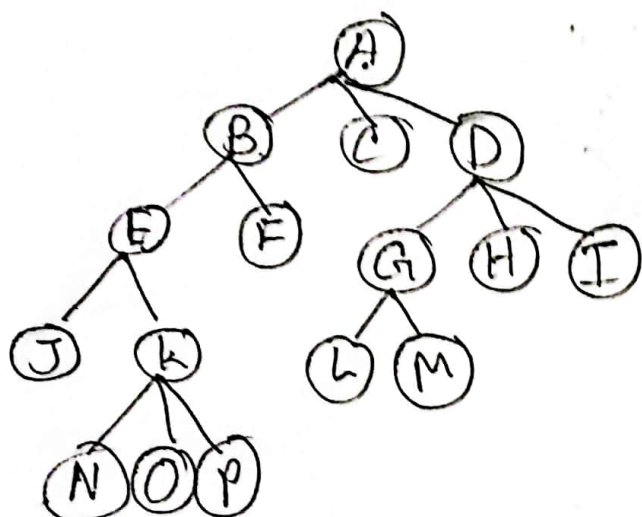
* post Order $\rightarrow$ ~~most~~, left, right, children $\rightarrow$ $\left[\begin{array}{l} \text{জানেন মনে} \\ \text{মাকে মিলিয়ে দেবে} \end{array} \right]$



~~Binary~~ → left, right root

* Inorder → left, root, right

Inorder (Left, Root, other children)



= J E N K O P B F A C L
D G M H I

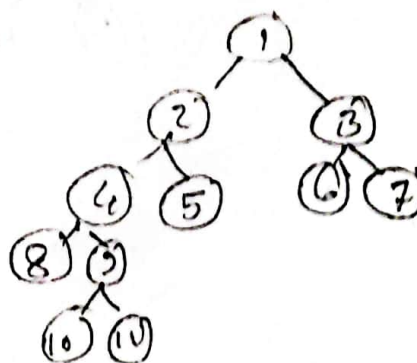
Level/Order

Draw Tree

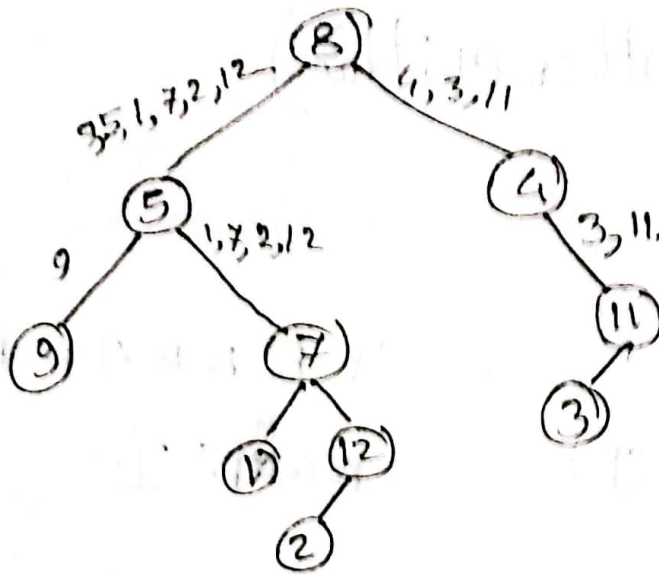
* In order → 8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7

left root from pre. Right

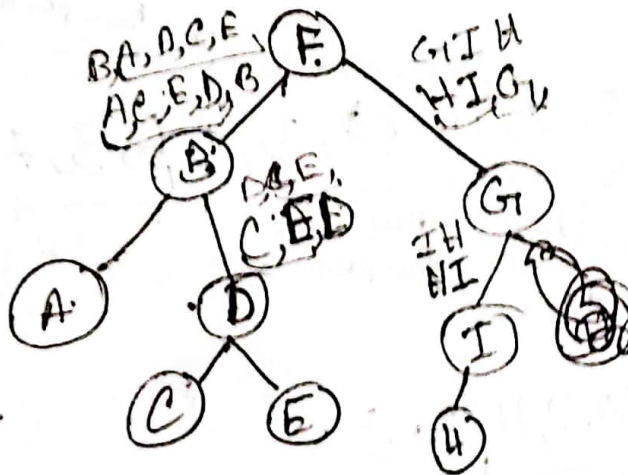
8, 4, 10, 9, 11, 2, 5



Post: 9, 1, 2, 12, 7, 5, 3, 11, 4, 8
 * Inorder $\rightarrow$ 9, 5, 1, 7, 2, 12, 8, 4, 3, 11
 (8) is marked as root

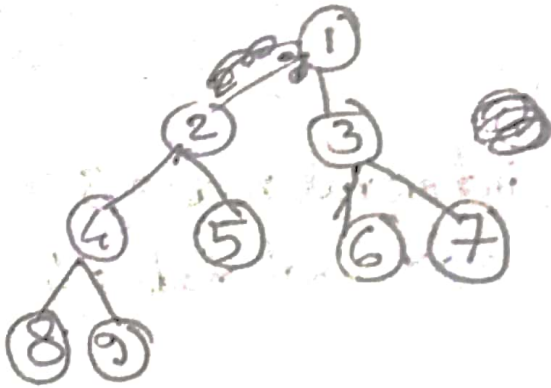


* Pre $\rightarrow$ F, B, A, D, C, E, G, I, H
 Post $\rightarrow$ A, C, E, D, B, H, I, G, (F)
 (F) is marked as root

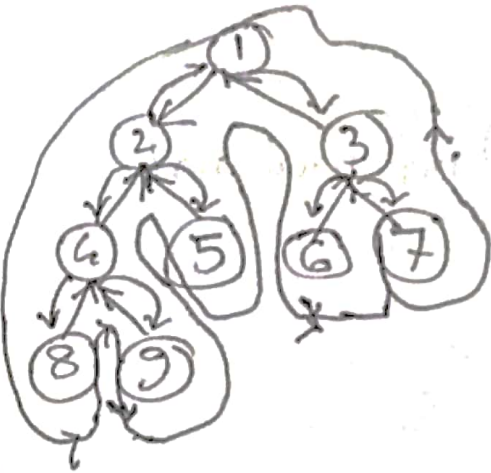


DSA-I

Euler Tour Traversal

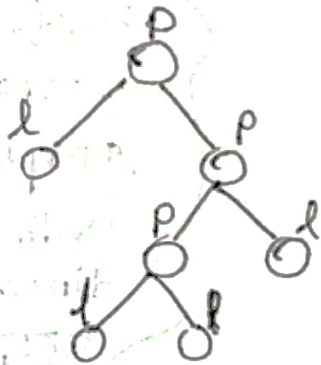


Directed graph
is a graph where, Undirected
graph is a graph where,
Undirected graph
directed graph is
a graph.



Every path is a path
is a path. Every path
is a path. Euler formula
follows the rule

$P(n)$: A proper binary tree with n internal nodes, has $n+1$ external nodes (leaves)



internal node = 3
external node = 4

Basis Step:

$P(0)$: A proper binary tree with 0 internal node has 0+1 external nodes.

0

internal node = 0
external node = 1

Inductive Step:

Let us assume $P(k)$ to be true.

$P(k)$: A proper binary tree with k internal nodes, has $k+1$ external nodes.

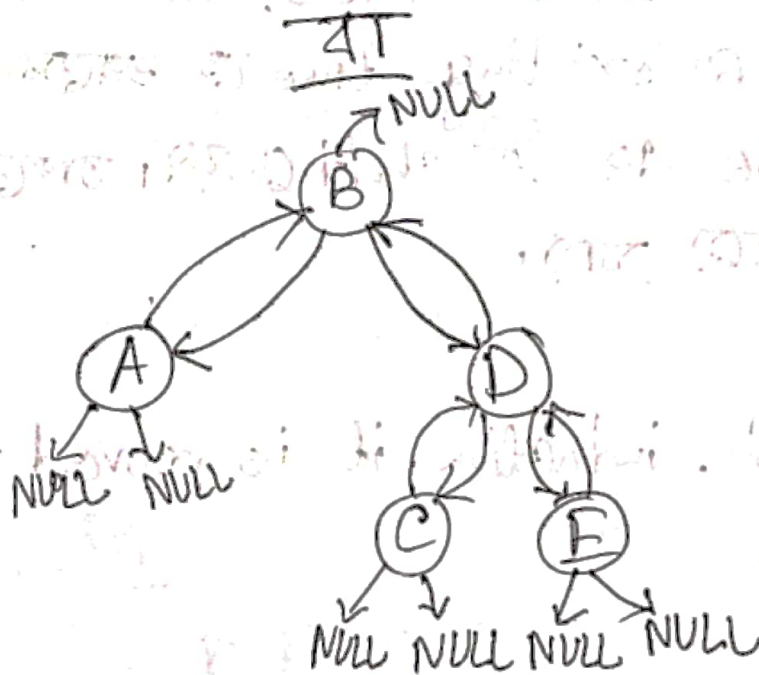
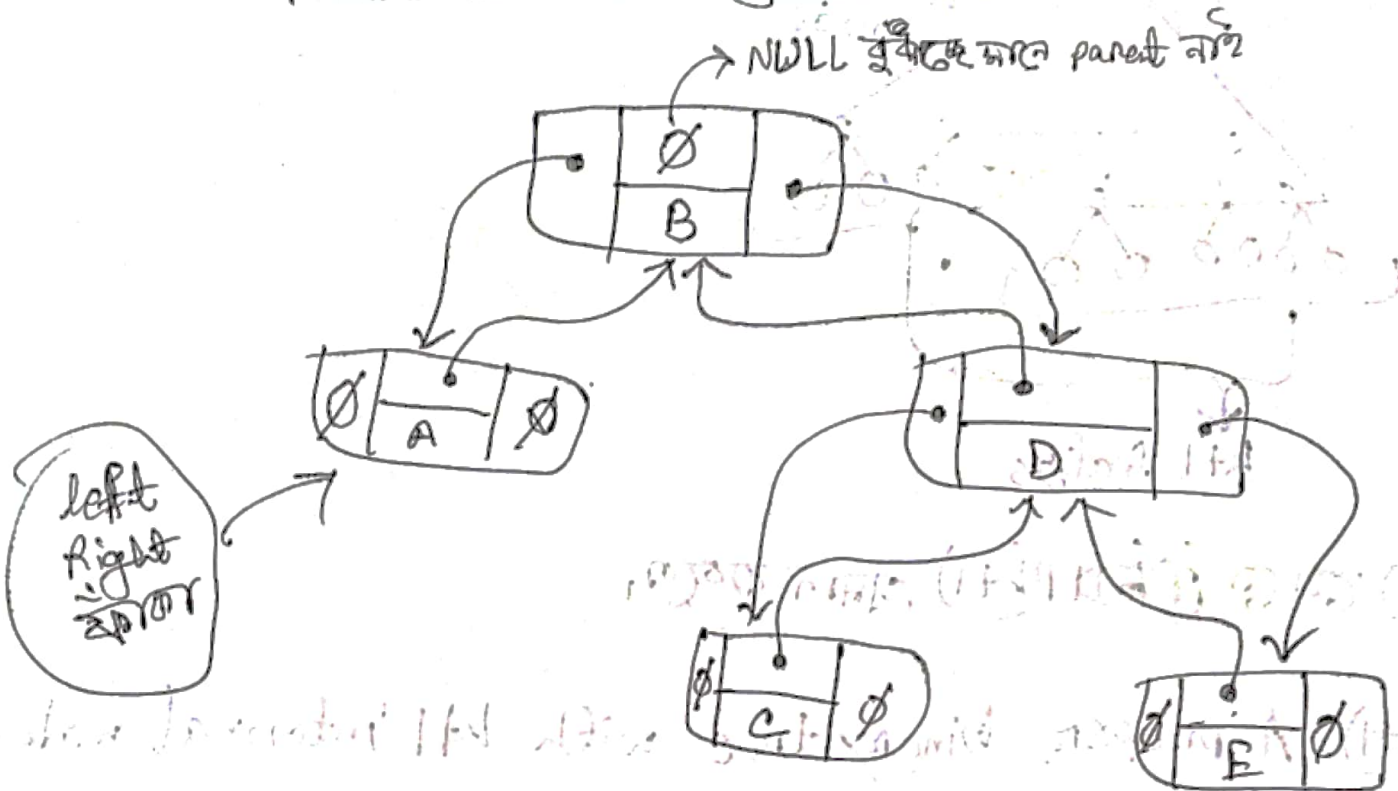


1874

A handwritten signature in dark ink, appearing to read "G. A." or similar initials, located at the bottom right of the page.

— — — — —

Implementing Binary Tree



```
struct TreeNode {
```

```
    char* value;
```

```
    struct TreeNode* parent;
```

```
    struct TreeNode* left;
```

```
    struct TreeNode* right;
```

```
};
```

~~etc~~

```
void Func(char*)
```

```
{
```

```
    struct TreeNode* newNode1 = malloc(sizeof
```

```
    (struct TreeNode));
```

```
    newNode1->value = "x";
```

```
    newNode1->left = NULL;
```

```
    newNode1->right = NULL;
```

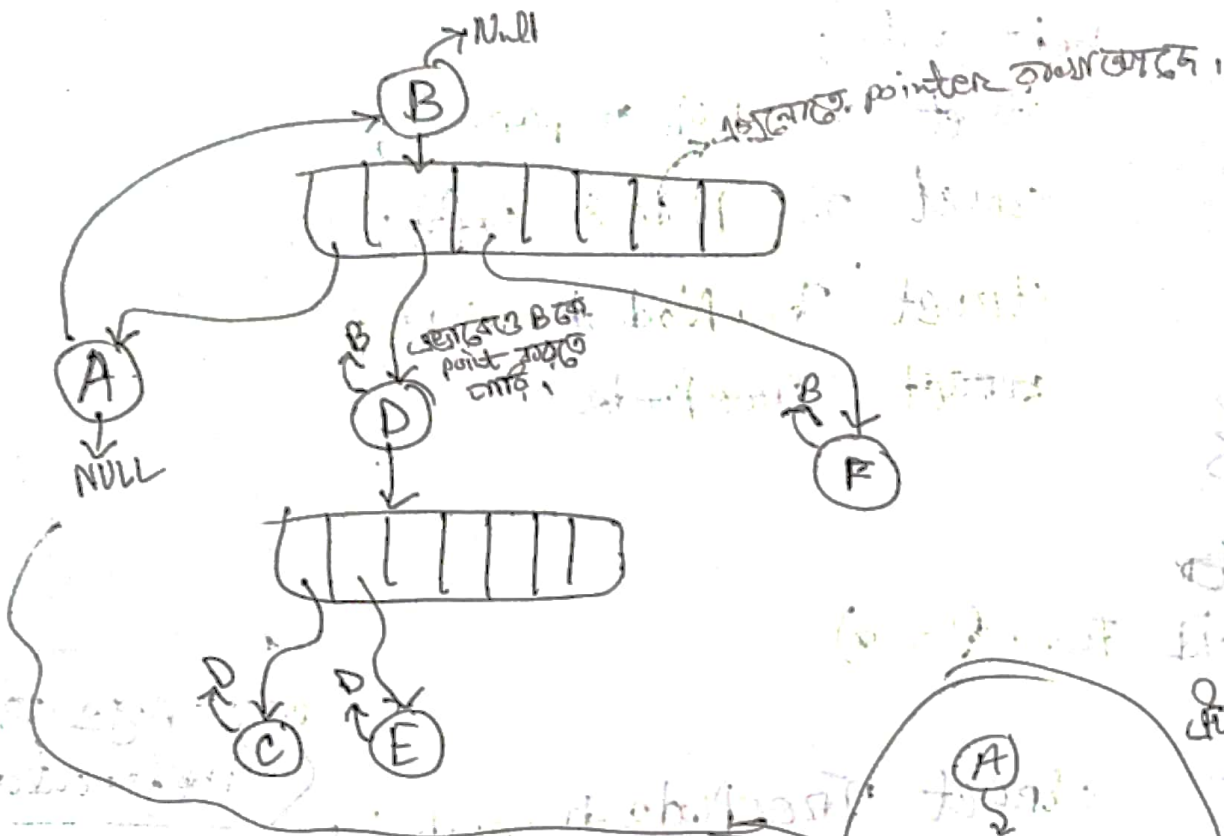
```
    temp = root->left;
```

```
    temp->left = newNode1;
```

```
    newNode1->parent = temp;
```

no page no. to be
implementation.

Implementing General Trees



func()

new Node 1 = malloc(---);

new Node 1 → value = 'X';

new Node 1 → parent = NULL;

new Node 1 → childrenContainer → childrenArray[0] = NULL

temp = root → childrenContainer → childrenArray[0];

new Node 1 → parent = temp;

temp → childrenContainer = malloc(sizeof(struct container));

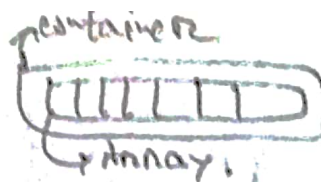
temp → childrenContainer → childrenArray[0] = new Node;

temp → childrenContainer → no of children ++;

```

newNode2 = malloc();
newNode2 → value = 'E';
newNode2 → parent = temp;
newNode2 → childrenContainer = NULL;
temp → childrenContainer → ChildrenArray[no. of Children] =
newNode2;
temp → childrenContainer → count ++;

```



Binary Search Tree

(Recursively)

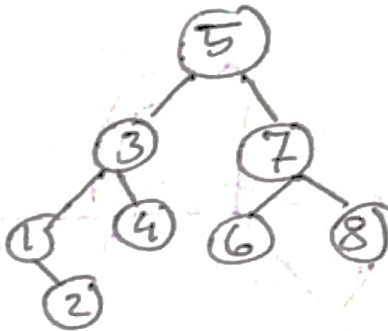
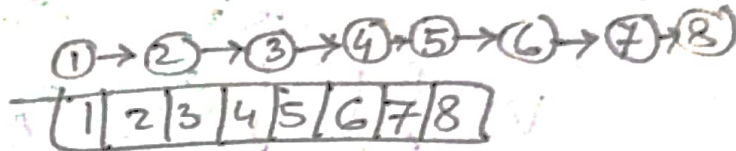
RTreeSearch(struct TreeNode *x, skey):

1. if ($x \neq \text{NULL} \parallel x \rightarrow \text{key} = \text{skey}$)
2. return x.
3. if ($x \rightarrow \text{key} < \text{skey}$)
4. return RTreeSearch($x \rightarrow \text{right}$, skey)
5. if (~~skey~~ $x \rightarrow \text{key} > \text{skey}$)
6. return RTreeSearch($x \rightarrow \text{left}$, skey)



DSA-I

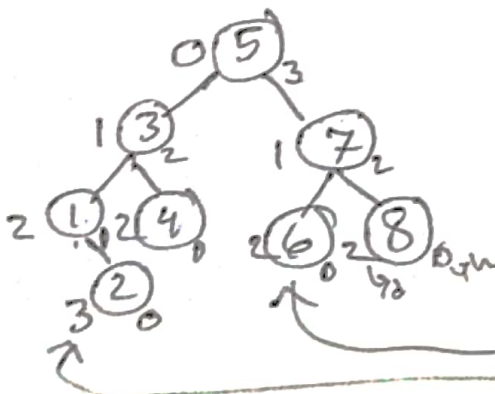
side 59 Binary Tree তে insert, min, max, delete search, এমন করে
 $O(h)$ ^{height} সময় লাগে। ফলে এটি Time কম লাগে বলে অনেক
 বনতে পারে।



Balanced Tree,

Tree এর height যদি h হয়।

leaf গুলোর depth $h, h-1$ হবে হবে।



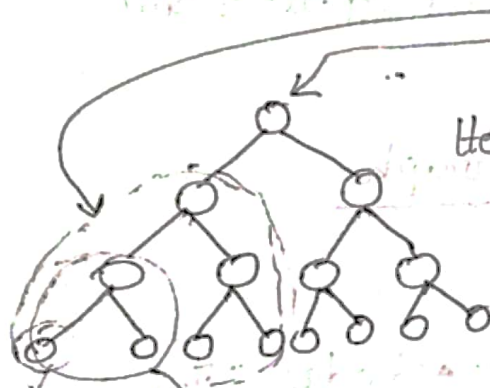
$h = 4$ ✓

$h-1 = 3$ ✓

depth = 2, 3 ✓

Tree Balanced 2141 Time complexity $\rightarrow$ $h = O(\log n)$

$h = O(\log n)$ $\Rightarrow$ $O(h)$



$$\text{Height}(h) = \text{Height}(1) + \text{Height}\left(\frac{n}{2}\right)$$

$$= 1 + \text{Height}\left(\frac{n}{2}\right) + 1$$

$$= 2 + \text{Height}\left(\frac{n}{4}\right)$$

$$= 3 + \text{Height}\left(\frac{n}{8}\right)$$

$$= 4 + \text{Height}\left(\frac{n}{16}\right)$$

$$= k + \text{Height}\left(\frac{n}{2^k}\right)$$

$$= k + \text{Height}(1)$$

Height = 0

$$= k + 0$$

$$= \log_2(n)$$

$$\therefore h = O(\log(n))$$

$$\frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

$$\Rightarrow \log_2(n) = \log_2 2^k$$

$$\therefore k = \log_2(n)$$

BST
C.T

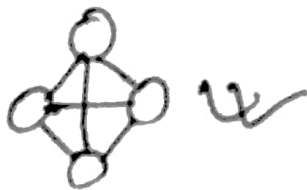
- statement (pseudo code) $\rightarrow$ { specific task by BST, in general function can do it }
- Insert(z)
- Delete(root $\rightarrow$ right $\rightarrow$ left)
- Successor, Predecessor

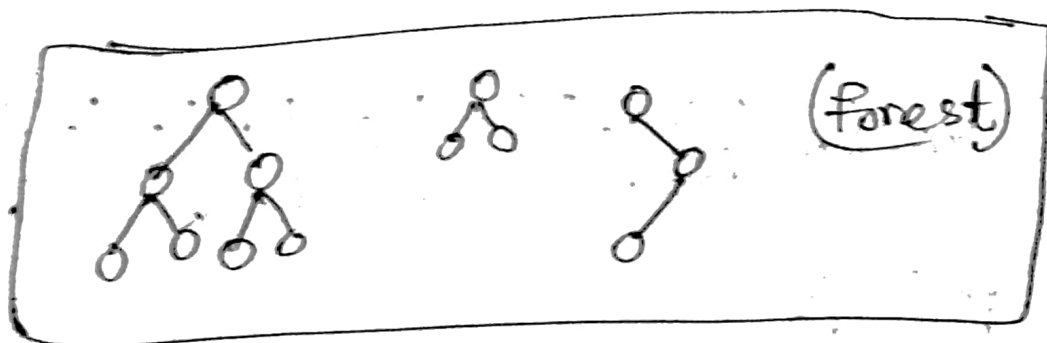
Graph

Dense graph $\rightarrow$ edge total

sparse graph $\rightarrow$ edge less. (slide)

* complete graph $\rightarrow$ every vertex is connected to every other vertex.





Undirected Graph
↓
Symmetric

Directed graph
↓
not Symmetric

$|V| + |E| \rightarrow \text{Directed}$

$|V| + 2|E| \rightarrow \text{Undirected}$

* slide-37 (color graph)

* struct Properties {

int score;

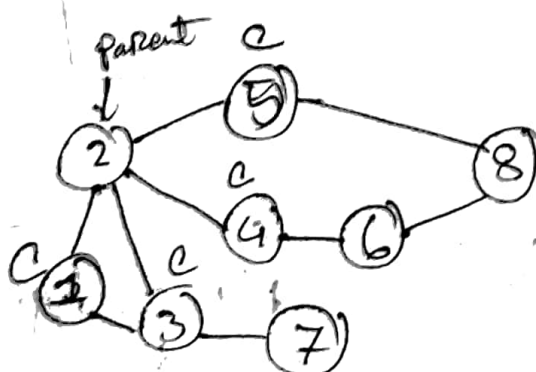
int color;

int parent;

}

array of structure

score	color	parent
-	-	-
-	-	-
-	-	-
-	-	-
-	-	-
-	-	-
-	-	-



	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1				1		
4		1			1			
5		1						1
6				1			1	
7								
8					1	1		

for(int i=1; i<=8; i++)
if (Adj[i][i] == -1)

score[i] = rand();
color[i] = rand();
parent[i] = 2;

CT-5 Wk-6 Tuesday